

assignment-03-code

September 10, 2026

1 Assignment 03

1.1 Import External Libraries

```
[1]: import numpy as np
import sympy as sp

from IPython.display import display, Math
```

1.2 2. Finding Eigenvectors Using Python

```
[2]: sigmay = np.array([[0, -1j], [1j, 0]])

display(Math(f"\sigma_y = {sp.latex(sp.Matrix(sigmay))}"))

print("-----")

eigenvalues, eigenvectors = np.linalg.eig(sigmay)
eigenvalues = np.round(eigenvalues, 4)
eigenvectors = np.round(eigenvectors, 4)

display(Math(f"\text{{Eigenvalues: }} {sp.latex(sp.Matrix(eigenvalues))}"))
display(Math(f"\text{{Eigenvectors: }} {sp.latex(sp.Matrix(eigenvectors))}"))
```

$$\sigma_y = \begin{bmatrix} 0 & -1.0i \\ 1.0i & 0 \end{bmatrix}$$

$$\text{Eigenvalues: } \begin{bmatrix} 1.0 \\ -1.0 \end{bmatrix}$$

$$\text{Eigenvectors: } \begin{bmatrix} -0.7071i & 0.7071 \\ 0.7071 & -0.7071i \end{bmatrix}$$

1.3 3. Pauli Matrix Multiplication by Hand and Using Python

```
[3]: sigmay = np.array([[0, -1j], [1j, 0]])

display(Math(f"\sigma_y = {sp.latex(sp.Matrix(sigmay))}"))
display(Math(f"\sigma_y^2 = {sp.latex(sp.Matrix(np.matmul(sigmay, sigmay)))}"))
```

$$\sigma_y = \begin{bmatrix} 0 & -1.0i \\ 1.0i & 0 \end{bmatrix}$$

$$\sigma_y^2 = \begin{bmatrix} 1.0 & 0 \\ 0 & 1.0 \end{bmatrix}$$

1.4 4. Pauli Matrix Multiplication by Hand and Using Python

```
[4]: sigmaz = np.array([[1, 0], [0, -1]])

display(Math(f"\sigma_z = {sp.latex(sp.Matrix(sigmaz))}"))
display(Math(f"\sigma_z^2 = {sp.latex(sp.Matrix(np.matmul(sigmaz, sigmaz)))}"))
```

$$\sigma_z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

$$\sigma_z^2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

1.5 5. Proof of Hermitianity

```
[5]: sigmay_dagger = sigmay.conj().T
display(Math(f"\sigma_y^{*T} = {sp.latex(sp.Matrix(sigmay_dagger))}"))

print("-----")

print(f"Check if Sigma Y is Hermitian: {np.allclose(sigmay_dagger, sigmay)}")
```

$$\sigma_y^{*T} = \begin{bmatrix} 0 & -1.0i \\ 1.0i & 0 \end{bmatrix}$$

Check if Sigma Y is Hermitian: True

1.6 6. Operation of Paul Matrix on Bra and Ket

```
[6]: zerospin = np.array([[1], [0]])
print("Dimension of zerospin:\n", zerospin.shape)

print("-----")

sigma_y_zero = np.dot(sigmay, zerospin)
display(Math(f"\sigma_y \ket{0} = {sp.latex(sp.Matrix(sigma_y_zero))}"))

bra_zero = zerospin.conj().T
bra_zero_sigma_y = np.dot(bra_zero, sigmay)
display(Math(f"\bra{0} \sigma_y = {sp.latex(sp.Matrix(bra_zero_sigma_y))}"))
```

Dimension of zerospin:
(2, 1)

$$\sigma_y |0\rangle = \begin{bmatrix} 0 \\ 1.0i \end{bmatrix}$$

$$\langle 0 | \sigma_y = [0 \quad -1.0i]$$
